

Tiny Link Hints Security Review

The entire extension source is included below. It is 98 total lines across all runtime, options, and manifest files, so it can be reviewed line by line in 15 minutes or less.

```
10 manifest.json
71 hints.js      (59 nonblank)
1  background.js
1  options.html
15 options.js    (14 nonblank)
```

Security posture:

- No dependencies or build step.
- No remote code, images, CSS, fonts, analytics, telemetry, or network requests.
- Stores one local preference: the selected modifier key.
- Sends one message type inside the extension: a user-selected link URL to open in a tab.
- Does not read form values, cookies, tokens, localStorage, sessionStorage, or page text.

hints.js

This is the main file. The first block is setup plus the only event listener.

```
const HINT_CHARS = "fjdkslaurieowhgmvcn";
const TARGETS = "a[href],button,input,textarea,select,[role=textbox]";

let hints = [], typed = "", action = "j", modifier = "meta";

chrome.storage.local.get({ mod: modifier }, ({ mod }) => modifier = mod);
chrome.storage.onChanged.addListener(({ mod }) => mod && (modifier = mod.newValue));

addEventListener("keydown", (event) => {
  if (handleShortcut(event)) return;
  if (!hints.length) return;
  if (event.key == "Escape") return event.preventDefault(), clearHints();
  if (event.key == "Backspace") typed = typed.slice(0, -1);
  else if (HINT_CHARS.includes(event.key.toLowerCase())) typed += event.key.toLowerCase();
  else return;
  event.preventDefault();
  updateMatches();
}, true);
```

The rest of the file is helper functions used by that listener.

```

function handleShortcut(event) {
  const key = event.code.slice(3).toLowerCase();
  if (!modifierIsDown(event) || !"jkl".includes(key)) return false;
  event.preventDefault();
  hints.length ? clearHints() : showHints(key);
  return true;
}

function showHints(key) {
  clearHints(); typed = ""; action = key;
  const targets = [...document.querySelectorAll(TARGETS)].filter(isVisible);
  const size = Math.max(1, Math.ceil(Math.log(targets.length) / Math.log(HINT_CHARS.length)));
  targets.forEach((target, index) => addHint(target, makeLabel(index, size)));
}

function addHint(target, id) {
  const box = target.getBoundingClientRect(), hint = document.createElement("b");
  hint.textContent = id.toUpperCase();
  hint.style.cssText = `position:fixed;z-index:2147483647;left:${Math.max(0,
box.left)}px;top:${Math.max(0, box.top + box.height /
2)}px;transform:translateY(-50%);background:#fff;color:#000;border:1px solid #000;padding:1px 3px;font:700
12px monospace`;
  document.documentElement.appendChild(hint);
  hints.push({ id, target, hint });
}

function updateMatches() {
  const matches = hints.filter((h) => h.id.startsWith(typed));
  hints.forEach((h) => h.hint.style.opacity = h.id.startsWith(typed) ? 1 : .25);
  if (matches.length == 1 && matches[0].id == typed) activate(matches[0].target), clearHints();
}

function makeLabel(n, size, label = "") {
  for (; size--> 0; n = Math.floor(n / HINT_CHARS.length)) label = HINT_CHARS[n % HINT_CHARS.length] + label;
  return label;
}

function isVisible(el) {
  const r = el.getBoundingClientRect();
  return r.width && r.height && r.bottom > 0 && r.right > 0 && r.top < innerHeight && r.left < innerWidth;
}

function modifierIsDown(event) {
  const key = modifier.replace(/\\d/, "");
  return key == "os" ? event.metaKey : event[key + "Key"];
}

function activate(el) {
  action == "j" || !el.href ? (el.focus(), el.click()) : chrome.runtime.sendMessage({ url: el.href,
active: action == "l" });
}

function clearHints() {
  hints.forEach((h) => h.hint.remove());
  hints = [];
}

```

In plain English: this labels visible links/buttons/inputs, lets the user type a label, then clicks/focuses that selected element or opens that selected link in a tab.

manifest.json

```
{
  "manifest_version": 2,
  "name": "Tiny Link Hints",
  "version": "0.1.0",
  "description": "Press a modifier plus J/K/L to label visible links.",
  "permissions": ["tabs", "storage"],
  "background": { "scripts": ["background.js"] },
  "options_ui": { "page": "options.html" },
  "content_scripts": [{ "matches": ["<all_urls>"], "js": ["hints.js"] }]
}
```

This is mostly WebExtension boilerplate. The permissions are limited to `storage` for the modifier preference and `tabs` for opening the user-selected link.

background.js

```
chrome.runtime.onMessage.addListener((m) => chrome.tabs.create({ url: m.url, active: m.active }));
```

One-line support script: opens the selected link in a tab.

options.html

```
<!doctype html><meta charset=utf-8><title>Tiny Link Hints</title><script src=options.js defer></script>
```

Boilerplate options page shell.

options.js

```
const groups = {
  Mac: { meta: "Command", alt: "Option", ctrl: "Control", shift: "Shift" },
  "Windows / Linux": { os: "Windows", alt2: "Alt", ctrl2: "Control", shift2: "Shift" },
};

chrome.storage.local.get({ mod: "meta" }, ({ mod }) => {
  for (const group in groups) {
    document.body.append(Object.assign(document.createElement("h2"), { textContent: group }));
    for (const value in groups[group]) {
      const checked = value === mod ? "checked" : "";
      document.body.insertAdjacentHTML("beforeend", `<label><input name=mod type=radio value=${value}
${checked}> ${groups[group][value]}</label><br>`);
    }
  }
  onchange = (event) => chrome.storage.local.set({ mod: event.target.value });
});
```

Small options UI: renders one radio group and saves the selected modifier key. It does not read web pages.

Bottom Line

This is a small local keyboard-navigation extension. The source shown above contains no path for data collection, remote communication, telemetry, or credential access.